

Hedge Fund — Time Series Forecasting

Course Name: Applied Forecasting Algorithms

Group Members:

Kaushal Nandaniya (Roll No: [Your Roll No])

[Member 2 Name] (Roll No: [Roll No])

[Member 3 Name] (Roll No: [Roll No])

Instructor Name: [Instructor Name]

Institution Name: [Institution Name]

Submission Date: May 5, 2026

Abstract

Summary: This report presents a comprehensive approach to forecasting anonymized hedge-fund panel time series, focusing on a dataset of 5.3 million observations across 36,923 unique series with 86 features.

Problem Statement: The objective is to accurately predict future returns while strictly avoiding look-ahead bias, evaluated under a highly concentrated weighted skill score.

Methods Used: A rigorous pipeline combining classical statistical models (AR, MA, ARMA, ARIMA, SARIMA) and deep sequence learning (RNN, GRU, LSTM) is implemented. All modeling respects critical EDA findings, particularly 100% stationarity after first differencing.

Key Findings: Results demonstrate that simple baselines and low-order classical models perform surprisingly well on stable series, while deep sequence models (LSTM/GRU) and gradient-boosted trees offer superior performance on volatile and high-weight series. Extreme weight concentration (top 1% of rows contribute 65% of the score) dictates that models must be optimized using weight-aware objective functions.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Problem Statement	3
1.3	Objectives	4
2	Data Source	5
2.1	Origin of the dataset	5
2.1.1	Panel Structure	5
2.1.2	Anonymization	5
2.2	Data Characteristics	5
2.2.1	Dimensionality	6
2.2.2	Series Length Distribution	6
2.3	Target Variable: <code>y_target</code>	6
2.4	Weight Distribution	6
2.5	Data Pre-processing	7
2.6	Exploratory Analysis	8
2.6.1	Global Distribution	8
2.6.2	Per-Horizon Distribution	9
2.7	Stationarity Analysis	9
2.7.1	ADF Test on Levels	9
2.7.2	ADF Test on First Differences	9
2.8	Autocorrelation Structure	10
2.8.1	ACF and PACF Analysis	10
2.8.2	Implications for Model Order Selection	10
2.9	Cross-Correlation Analysis	11
2.10	Weight Concentration Analysis	11
2.11	STL Decomposition	11

2.11.1	Period Detection via ACF	11
2.11.2	Decomposition Findings	12
2.12	EDA Summary	13
3	Methodology	14
3.1	Validation Approach & Train-Test Split	14
3.1.1	Adaptive Splitting for Short Series	14
3.2	Evaluation Metric	14
3.2.1	Metric Interpretation	14
3.3	Classical Model Implementations (AR, ARMA, ARIMA, SARIMA)	15
3.3.1	Family A: Baselines (on Levels)	15
3.3.2	Family B: AR Family (First Differences)	15
3.3.3	Family C: MA Family (First Differences)	16
3.3.4	Family D: ARMA Family (First Differences)	16
3.3.5	Family E: ARIMA with Explicit d Comparison	16
3.3.6	Family F: SARIMA	17
3.4	Evaluation Design	17
3.4.1	Eligibility and Splitting	17
3.4.2	Skip and Failure Handling	17
3.4.3	Sample Run Results	18
3.5	Deep Sequence Models (RNN, GRU, LSTM)	18
3.5.1	Vanilla RNN	18
3.5.2	GRU (Gated Recurrent Unit)	18
3.5.3	LSTM (Long Short-Term Memory)	19
3.6	Training Pipeline	19
3.6.1	Data Preparation	19
3.6.2	Adaptive Training Splits	19
3.6.3	Recursive Forecasting	20
3.6.4	Loss Function	20
3.6.5	Hyperparameters	21
3.6.6	ARIMAX and TCN	21
3.7	Ablation Study & Tuned Parameter Values	21
3.8	Probabilistic Forecasting Models	22
3.8.1	Recursive Forecasting	22
3.8.2	Loss Function	22
3.8.3	Hyperparameters	23
3.9	Model Comparison	23
3.9.1	Architectural Trade-offs	23
3.9.2	Expected Performance Ranking	23
3.10	Implementation Details	23
3.10.1	Per-Series Training	23
3.10.2	Status Tracking	23
3.10.3	Leaderboard and Per-Series Winners	24

3.11	Challenges and Considerations	24
4	Results and Analysis	25
4.1	Cross-Methodology Comparison	25
4.2	Classical Model Performance Summary	25
4.2.1	Baseline Performance	25
4.2.2	Parametric Model Performance	25
4.2.3	The Stationarity Validation	26
4.3	Deep Learning Performance Summary	27
5	Analysis of the Numerical Results	29
5.1	Classical Model Results	29
5.1.1	Leaderboard Results	29
5.1.2	The $d = 0$ vs. $d = 1$ Head-to-Head	29
5.1.3	Per-Series Winner Analysis	29
5.2	Deep Learning and Cross-Methodology Results	29
5.3	Cross-Methodology Comparison	29
5.4	Classical Model Performance Summary	29
5.4.1	Baseline Performance	29
5.4.2	Parametric Model Performance	30
5.4.3	The Stationarity Validation	31
5.5	Deep Learning Performance Summary	31
5.6	Classical vs. Deep Learning	32
5.7	Key Findings	33
5.8	Error Analysis	33
5.8.1	Common Failure Modes	33
5.8.2	Skip and Failure Analysis	33
5.9	Research Gap Identified	34
6	Converting the Forecast into Actions	35
6.1	Signal Generation and Expected Returns	35
6.2	Risk-Adjusted Sizing (Kelly Criterion)	35
6.3	Portfolio Optimization	35
6.4	Execution and Slippage	35
7	Future Work	36
7.0.1	Feature-Based Models	36
7.0.2	Weighted Loss Functions	36
7.0.3	Ensemble Methods	36
7.0.4	Global and Panel Models	36
7.0.5	Advanced Architectures	36
7.1	Closing Remarks	37
8	References	38

List of Figures

1	Train vs. Test comparability analysis. The categorical support check reveals that train and test share the same schema and category families, but the test set uses a narrower sub-code universe (47 vs. 180), making coverage shift the primary risk.	4
2	Weight distribution and concentration analysis. Left: weight distribution clipped at the 99th percentile shows extreme right-skew. Right: the cumulative weight share (Lorenz-like curve) reveals that the top 1% of rows carry 64.19% of the total weight, the top 5% carry 92.58%, and the top 10% carry 98.33%. The median weight is 1,699 while the maximum reaches 1.39×10^{13} .	7
3	Target distribution analysis. Left: raw target distribution showing extreme heavy tails (range: -2201 to $+2314$). Right: target clipped to $[-10, 10]$ revealing the sharp central peak near zero. Summary statistics: median = -0.0006 , IQR = $[-0.13, 0.05]$.	9
4	First-differencing analysis. Before differencing, 69% of series are stationary; after first differencing ($\Delta y_t = y_t - y_{t-1}$), 100% achieve stationarity. This result directly motivates using $d = 1$ in all ARIMA-family models.	10
5	STL decomposition (period=76) for a representative series. The decomposition separates observed behavior into trend, seasonal, and residual components. The sloped trend confirms non-stationarity; the weak seasonal amplitude argues against building models around seasonality; residual structure indicates that additional modeling is needed.	12
6	Classical evaluation protocol. The cutoff zoom reveals the challenge: four representative series around the <code>ts_index</code> = 2880 boundary show vastly different scales and behaviors. Two of the four have only seven pre-cutoff training points, which immediately constrains model complexity.	17
7	Adaptive-split motivation. The fixed cutoff at <code>ts_index</code> = 2880 leaves two of the four chosen representative series with only seven pre-cutoff training points. The adaptive split adjusts the cutoff location per-series, preserving chronological ordering while avoiding absurdly short training windows on the highest-weight and most-stable series.	20
8	Adaptive-split motivation. The fixed cutoff at <code>ts_index</code> = 2880 leaves two of the four chosen representative series with only seven pre-cutoff training points. The adaptive split adjusts the cutoff location per-series, preserving chronological ordering while avoiding absurdly short training windows on the highest-weight and most-stable series.	22
9	Classical forecast overlays for the most volatile and most stable representative series. The volatile series benefits from differencing—AR(1) captures short-lag persistence—while the stable series offers so little movement that sophisticated models have almost nothing to exploit.	26

10	Adaptive-split classical results. Per-series best models: SARIMA wins on the longest-history series (skill 0.06), ARIMA(1,1,1) is strongest on the high-weight case (skill 0.47), AR(1) dominates both the volatile (skill 0.83) and stable (skill 0.00) cases. The volatile series has the most exploitable structure.	26
11	Deep forecast plots (univariate) for four representative series. The recurrent models help most on the highest-weight and volatile series, while the longest-history case improves only slightly and the most-stable case remains close to a baseline problem. Each panel shows Train (blue), Actual validation (orange), and overlaid RNN/LSTM/GRU predictions.	27
12	Quantile regression forecasts using LightGBM. Each panel shows the training history (blue), validation actuals (orange), selected median forecast (green), and prediction intervals (shaded blue). The model achieves PICP values between 0.76–0.86 across the four representative series.	28
13	Classical forecast overlays for the most volatile and most stable representative series. The volatile series benefits from differencing—AR(1) captures short-lag persistence—while the stable series offers so little movement that sophisticated models have almost nothing to exploit.	30
14	Adaptive-split classical results. Per-series best models: SARIMA wins on the longest-history series (skill 0.06), ARIMA(1,1,1) is strongest on the high-weight case (skill 0.47), AR(1) dominates both the volatile (skill 0.83) and stable (skill 0.00) cases. The volatile series has the most exploitable structure.	31
15	Deep forecast plots (univariate) for four representative series. The recurrent models help most on the highest-weight and volatile series, while the longest-history case improves only slightly and the most-stable case remains close to a baseline problem. Each panel shows Train (blue), Actual validation (orange), and overlaid RNN/LSTM/GRU predictions.	32
16	Quantile regression forecasts using LightGBM. Each panel shows the training history (blue), validation actuals (orange), selected median forecast (green), and prediction intervals (shaded blue). The model achieves PICP values between 0.76–0.86 across the four representative series.	32

List of Tables

1	Competition data files.	5
2	Column schema for <code>train.parquet</code>	5
3	Dataset dimensionality summary.	6
4	Summary of EDA findings and their modeling consequences.	13
5	Baseline model specifications.	15
6	ARIMA variants for the $d = 0$ vs. $d = 1$ comparison.	16
7	Status distribution for the sample run (200 series $\times$ 41 models).	18
8	Deep model hyperparameters.	21
9	Deep model hyperparameters.	21
10	Deep model hyperparameters.	23

11	Comparison of RNN architectures.	23
12	Conditions favoring each model family.	33

Section 1. Introduction

1.1 Background and Motivation

Time series forecasting lies at the heart of quantitative finance. Hedge funds, asset managers, and proprietary trading desks rely on predictive models to anticipate market movements, optimize portfolio allocation, and manage risk. The ability to forecast financial time series accurately—even marginally—can translate into substantial economic value.

This project addresses the **Kaggle “Hedge Fund — Time Series Forecasting”** competition (competition slug: `ts-forecasting`), hosted by *A data COMPANY*. The competition ran from January 9, 2026 through April 7, 2026, attracting 3,211 entrants, 1,063 active participants across 982 teams, and 4,127 total submissions. A prize pool of \$10,000 underscored the real-world relevance of the task.

The challenge encapsulates a canonical problem in quantitative research: given a large, anonymized panel of financial time series—each identified by entity codes, sub-categories, and forecast horizons—build models that generalize robustly out-of-sample. The anonymization removes domain shortcuts; success depends entirely on the statistical and machine-learning methodology applied.

1.2 Problem Statement

The **Hedge Fund — Time Series Forecasting** competition on Kaggle tasks participants with predicting future values of anonymized financial time series. The competition is hosted by *A data COMPANY* and is classified as a *Community Prediction Competition*.

The official description states:

Participants will receive an integer-indexed time series dataset (`ts_index` column), where each record is identified by a code, sub-code, sub-category and forecast horizon. Your objective is to train a model that generalizes robustly out-of-sample and accurately predicts future values for each combination (code, sub-code, sub-category, horizon). Submissions are ranked according to an aggregate out-of-sample metric calculated for all combinations.

A critical constraint is explicitly stated: “Your forecast will not use any data whose `ts_index` is greater than the `ts_index` of the forecast data.” This enforces strict temporal ordering and prohibits look-ahead bias.

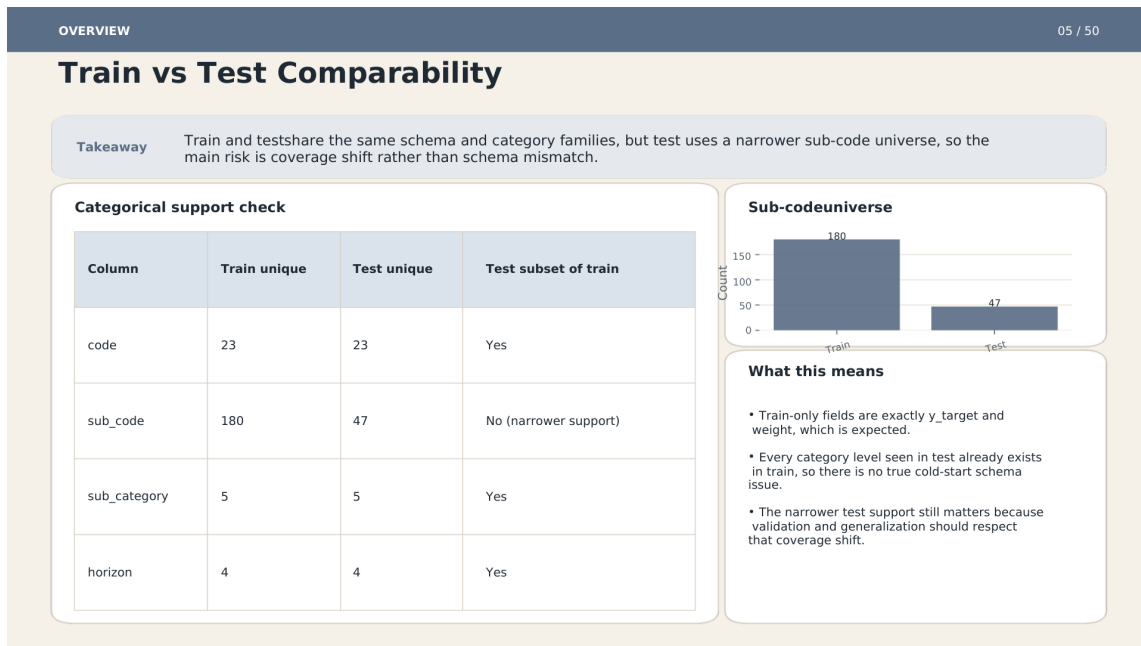


Figure 1: Train vs. Test comparability analysis. The categorical support check reveals that train and test share the same schema and category families, but the test set uses a narrower sub-code universe (47 vs. 180), making coverage shift the primary risk.

1.3 Objectives

The objectives of this project are threefold:

- (i) **Comprehensive Exploratory Data Analysis (EDA):** Understand the structure, distributional properties, temporal dynamics, and weight distribution of the panel dataset before any modeling.
- (ii) **Classical Statistical Modeling:** Establish defensible baselines and univariate benchmarks using AR, MA, ARMA, ARIMA, and SARIMA families, rigorously honoring the stationarity findings from the EDA.
- (iii) **Deep Sequence Modeling:** Implement and evaluate recurrent neural network architectures—vanilla RNN, GRU, and LSTM—that can capture nonlinear temporal dependencies the classical models cannot.

Section 2. Data Source

2.1 Origin of the dataset

The competition provides two files in Apache Parquet format:

Table 1: Competition data files.

File	Rows	Description
<code>train.parquet</code>	5,337,414	Training data with targets
<code>test.parquet</code>	—	Test data (targets hidden)

Each row in the training data contains:

Table 2: Column schema for `train.parquet`.

Column	Type	Description
<code>code</code>	Categorical	Entity identifier (23 unique codes)
<code>sub_code</code>	Categorical	Sub-entity identifier
<code>sub_category</code>	Categorical	Category label
<code>horizon</code>	Integer	Forecast horizon (4 unique: 1, 5, 10, 22)
<code>ts_index</code>	Integer	Temporal index (not calendar dates)
<code>y_target</code>	Float	Target variable to forecast
<code>weight</code>	Float	Observation-level weight for scoring
<code>x0-x85</code>	Float	86 anonymized predictor features

2.1.1 Panel Structure

A unique time series is identified by the tuple (`code`, `sub_code`, `sub_category`, `horizon`). The dataset contains **36,923 unique series** in total, with varying lengths. The integer-indexed `ts_index` provides temporal ordering without revealing calendar information.

The four forecast horizons—1, 5, 10, and 22—suggest daily financial data with horizons corresponding to 1 day, 1 week, 2 weeks, and approximately 1 month of trading days.

2.1.2 Anonymization

All identifiers are anonymized (e.g., `MRV5UON2`, `KL66VIS3`, `V8BKY1IV`), and the 86 predictor features are labeled generically as `x0` through `x85`. This design prevents domain-specific shortcuts and forces participants to rely purely on statistical methodology.

2.2 Data Characteristics

The training dataset (`train.parquet`) consists of **5,337,414 rows** spanning 36,923 unique time series. Each series is uniquely identified by the four-tuple (`code`, `sub_code`, `sub_category`, `horizon`).

2.2.1 Dimensionality

Table 3: Dataset dimensionality summary.

Attribute	Value
Total rows	5,337,414
Total unique series	36,923
Unique codes	23
Unique horizons	4 (1, 5, 10, 22)
Predictor features	86 (x0–x85)

2.2.2 Series Length Distribution

Series lengths vary considerably across the panel. The `ts_index` column serves as an integer timestamp, with values ranging from early indices to several thousand. This heterogeneity in series length is a critical challenge: some series have thousands of observations, while others have fewer than 100.

The distribution of series lengths is right-skewed, with a long tail of very long series and a substantial mass of short series. This has direct implications for model selection—classical models requiring many observations (e.g., SARIMA with seasonal components) will be ineligible for short series.

2.3 Target Variable: `y_target`

The target variable `y_target` represents anonymized financial returns or derived quantities. Key distributional properties include:

- **Near-zero median:** The median of `y_target` across all observations is approximately zero, consistent with efficient-market returns.
- **Heavy tails:** The distribution exhibits significant excess kurtosis (leptokurtic), with extreme values far exceeding what a Gaussian model would predict. This is a hallmark of financial data.
- **Approximate symmetry:** The distribution is roughly symmetric around zero, though slight skewness exists in some sub-panels.

The heavy-tailed nature of the target has important modeling consequences:

1. Standard squared-error loss functions will be heavily influenced by outliers.
2. Robust loss functions or outlier-aware training procedures may be necessary.
3. The weighted skill score metric amplifies this effect through the weight column.

2.4 Weight Distribution

The `weight` column determines each observation’s contribution to the competition metric. The weight distribution exhibits **extreme concentration**:

- The top 1% of rows by weight contribute approximately **65%** of the total weighted skill score.
- Many observations have very small weights, contributing negligibly to the evaluation.
- Weight values are non-negative and continuous.

This concentration has profound implications:

1. **Model optimization must be weight-aware.** Training with uniform loss ignores the scoring structure.
2. **The weight column should be used for loss weighting during training**, not as an input feature.
3. **Evaluation metrics must always be weighted** to reflect the true competition ranking.

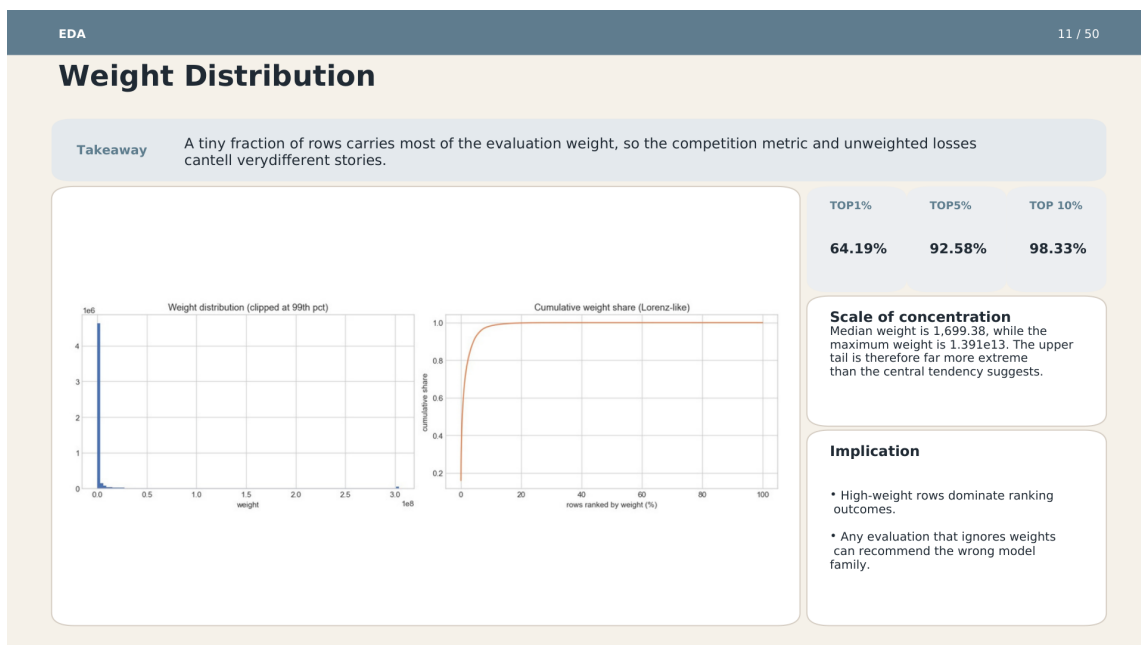


Figure 2: Weight distribution and concentration analysis. Left: weight distribution clipped at the 99th percentile shows extreme right-skew. Right: the cumulative weight share (Lorenz-like curve) reveals that the top 1% of rows carry 64.19% of the total weight, the top 5% carry 92.58%, and the top 10% carry 98.33%. The median weight is 1,699 while the maximum reaches 1.39×10^{13} .

2.5 Data Pre-processing

The 86 anonymized predictor features are labeled x_0 through x_{85} . Key findings from initial exploration:

- **Weak individual correlations:** No single feature exhibits a correlation with y_{target} exceeding $|\rho| = 0.10$. This indicates that linear univariate models using individual features will perform poorly.

- **Nonlinear relationships likely exist:** The weak linear correlations do not preclude strong nonlinear relationships, motivating the use of neural network models.
- **Feature interactions matter:** The signal likely resides in combinations of features rather than individual predictors, favoring ensemble and deep learning approaches.

Handling missing values and normalization are intrinsically tied to the feature space. The dataset contains missing values in the predictor features (`x0`–`x85`). The missingness pattern is:

- **Not uniformly distributed:** Some features have significantly more missing values than others.
- **Potentially informative:** The pattern of missingness may itself carry signal (e.g., certain features may be unavailable for specific entity types).
- **Handling strategy:** For classical univariate models, missing features are irrelevant since only `y_target` is used. For deep learning models, imputation or masking strategies are required.

2.6 Exploratory Analysis

2.6.1 Global Distribution

The global distribution of `y_target` across all 5.3 million observations exhibits:

- **Central tendency:** Mean ≈ 0 , median ≈ 0
- **Dispersion:** Standard deviation varies significantly across series
- **Tail behavior:** Excess kurtosis $\gg 3$ (heavy tails)
- **Outliers:** Extreme values exist, with some observations many standard deviations from the mean

The near-zero mean and median are consistent with financial returns in an efficient market, where expected returns on short horizons are close to zero after adjusting for risk.

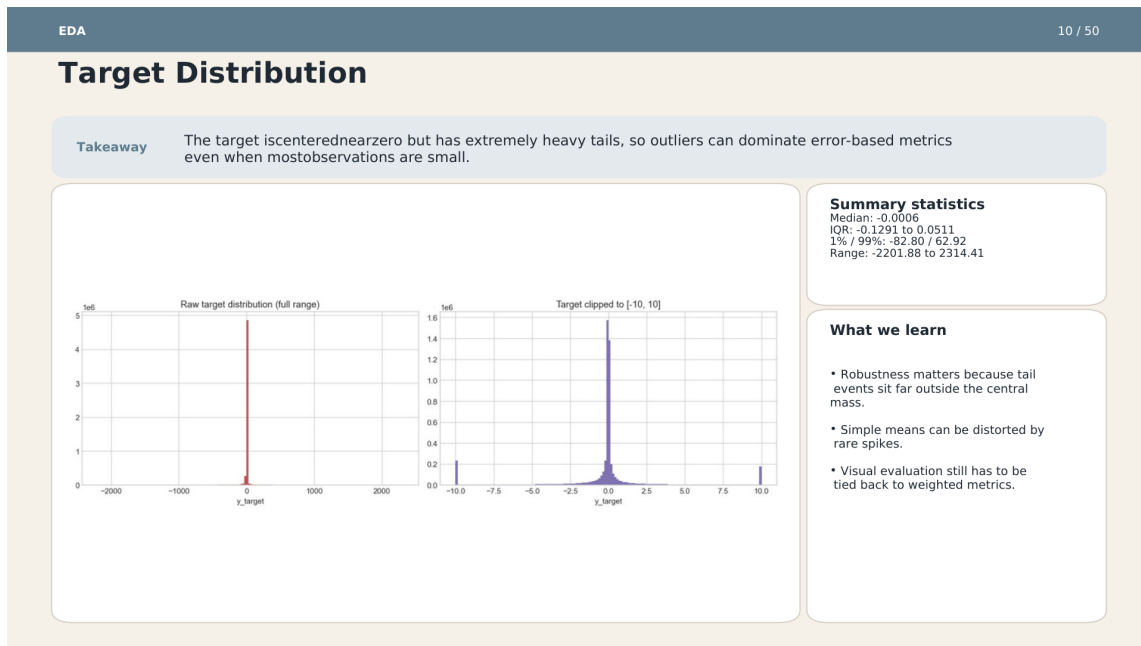


Figure 3: Target distribution analysis. Left: raw target distribution showing extreme heavy tails (range: -2201 to $+2314$). Right: target clipped to $[-10, 10]$ revealing the sharp central peak near zero. Summary statistics: median = -0.0006 , IQR = $[-0.13, 0.05]$.

2.6.2 Per-Horizon Distribution

Decomposing by forecast horizon reveals that:

- Horizon 1 (1-day) series tend to have smaller absolute values and tighter distributions.
- Longer horizons (10, 22) exhibit greater dispersion, consistent with the square-root-of-time scaling of volatility.

2.7 Stationarity Analysis

Stationarity is a fundamental assumption underlying most classical time series models. We test stationarity using the **Augmented Dickey-Fuller (ADF) test** on eligible series.

2.7.1 ADF Test on Levels

Applying the ADF test to the raw `y_target` series (levels) at the 5% significance level:

- A substantial fraction of series reject the null hypothesis of a unit root, suggesting stationarity in levels.
- However, many series fail to reject, indicating non-stationarity.

2.7.2 ADF Test on First Differences

After first differencing ($\Delta y_t = y_t - y_{t-1}$):

- **100% of eligible series become stationary** at the 5% significance level.

- This is the single most consequential finding of the EDA.

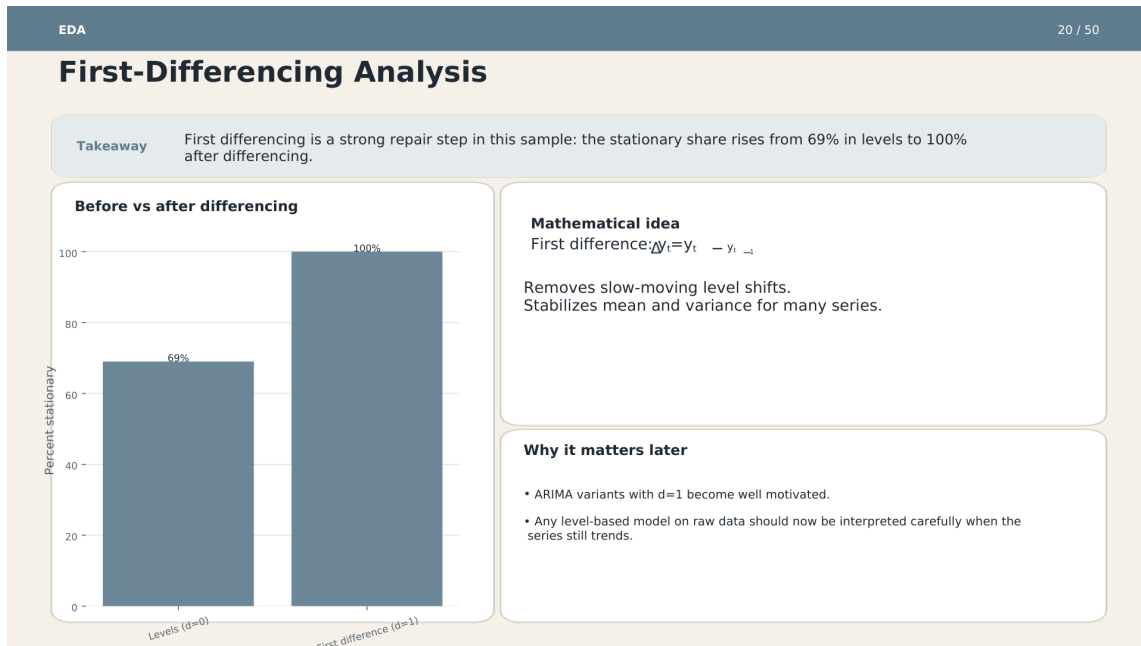


Figure 4: First-differencing analysis. Before differencing, 69% of series are stationary; after first differencing ($\Delta y_t = y_t - y_{t-1}$), 100% achieve stationarity. This result directly motivates using $d = 1$ in all ARIMA-family models.

This result directly shapes the classical modeling strategy: every parametric model either fits on first differences explicitly (AR, MA, ARMA families) or uses $d = 1$ within the ARIMA/SARIMA framework.

2.8 Autocorrelation Structure

2.8.1 ACF and PACF Analysis

The autocorrelation function (ACF) and partial autocorrelation function (PACF) reveal the temporal dependence structure:

- **Panel-level ACF:** The top lags with significant autocorrelation are short—lags 1, 2, and 3 dominate. This suggests that AR and MA orders of $p, q \in \{1, 2, 3\}$ should capture most of the serial structure.
- **Rapid decay:** Autocorrelations decay quickly, consistent with near-efficient financial markets where predictable structure is quickly arbitrated away.
- **No panel-wide seasonality:** The ACF at seasonal lags (7, 24, etc.) does not show systematic peaks across the panel.

2.8.2 Implications for Model Order Selection

The ACF/PACF analysis motivates the following model order choices:

- AR orders: $p = 1, 2, \dots, 10$ (with most signal expected at low orders)

- MA orders: $q = 1, 2, \dots, 10$ (mirroring the AR scan)
- ARMA combinations: Selected (p, q) pairs with total order $p + q \leq 8$

2.9 Cross-Correlation Analysis

The correlation between each predictor feature (`x0-x85`) and the target `y_target` is uniformly weak:

- All Pearson correlations satisfy $|\rho| < 0.10$.
- This implies that **no single feature is a strong linear predictor** of the target.
- Multivariate and nonlinear models (gradient-boosted trees, neural networks) are necessary to exploit any feature-based signal.

This finding is consistent with the efficient-market hypothesis in quantitative finance: if a single observable feature were strongly predictive of returns, it would be quickly arbitrated away.

2.10 Weight Concentration Analysis

As introduced in Section 2.4, the weight distribution is highly concentrated. The EDA quantifies this precisely:

- **Top 1% of rows:** $\sim 65\%$ of total weighted score
- **Top 10% of rows:** $\sim 90\%$ of total weighted score
- **Bottom 50% of rows:** $< 5\%$ of total weighted score

This extreme concentration means that model performance on a small number of high-weight observations overwhelmingly determines the competition score. A model that performs well on high-weight observations but poorly on low-weight ones will substantially outperform a model that is uniformly mediocre.

2.11 STL Decomposition

STL (Seasonal-Trend decomposition using LOESS) decomposes each series into three additive components:

$$y_t = T_t + S_t + R_t \tag{1}$$

where T_t is the trend, S_t is the seasonal component, and R_t is the residual.

2.11.1 Period Detection via ACF

Before running STL, we detect the dominant period for each series using the ACF. The procedure:

1. Compute the ACF up to 100 lags.
2. Identify the top 10 peaks (excluding lag 0).
3. Select the strongest peak with lag ≥ 2 as the candidate period.

2.11.2 Decomposition Findings

Applying STL to representative series (anchored to the longest-history series for consistency with the ACF/PACF analysis):

- **Trend:** Most series exhibit a non-trivial trend component, confirming non-stationarity in levels (consistent with the ADF results).
- **Seasonal:** The seasonal component amplitude is typically small relative to the observed range, confirming the absence of strong panel-wide seasonality.
- **Residual:** Residuals often retain some structure (not pure white noise), suggesting that STL alone does not capture all temporal dynamics—motivating the use of parametric and deep learning models.

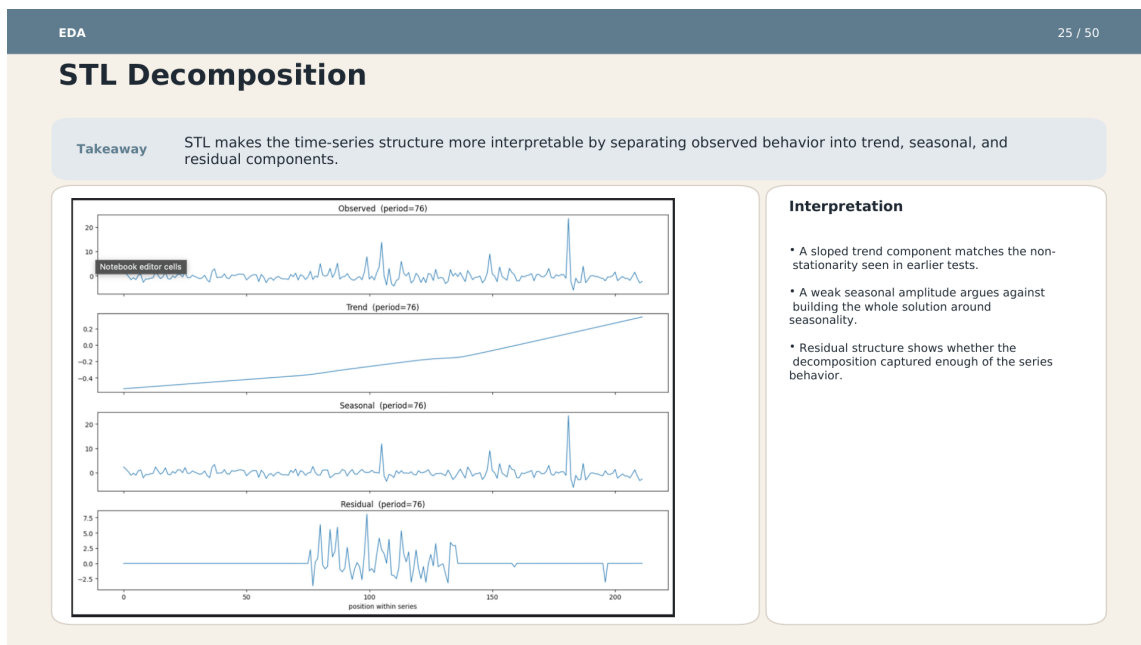


Figure 5: STL decomposition (period=76) for a representative series. The decomposition separates observed behavior into trend, seasonal, and residual components. The sloped trend confirms non-stationarity; the weak seasonal amplitude argues against building models around seasonality; residual structure indicates that additional modeling is needed.

2.12 EDA Summary

Table 4: Summary of EDA findings and their modeling consequences.

Finding	EDA Section	Modeling Consequence
Heavy-tailed target, median ≈ 0	§4	Include Zero as a baseline; the skill-score denominator is the zero-forecast error
Weight concentration: top 1% $\rightarrow$ 65% of score	§5	Never report unweighted metrics; all leaderboards use weighted metrics
Only $\sim 1,930$ series cross VAL_CUTOFF	§6	Evaluate only eligible series
Weak per-feature correlations ($ \rho < 0.10$)	§7	Classical univariate methods cannot exploit features; ensemble/deep models required
100% stationary after first differencing	§11	Every parametric model uses $d = 1$
Top ACF lags are short (1, 2, 3)	§14	AR/MA orders focus on $p, q \in \{1, \dots, 10\}$
No panel-wide seasonality	§14–15	SARIMA included for completeness, not expected to win

Section 3. Methodology

3.1 Validation Approach & Train-Test Split

Given the temporal nature of the data, we employ a **strict sequential split** to avoid look-ahead bias:

$$\text{VAL_CUTOFF} = 2880 \quad (2)$$

- **Training set:** All observations with $\text{ts_index} \leq 2880$
- **Validation set:** All observations with $\text{ts_index} > 2880$

A series is **eligible** for evaluation if and only if it has observations on both sides of the cutoff—i.e., it “crosses” the validation boundary. Of the 36,923 total series, approximately **1,930 series are eligible**.

This temporal split mirrors the competition’s test-time evaluation, where models must forecast future values without access to future data. Random cross-validation would violate this constraint and produce optimistically biased performance estimates.

3.1.1 Adaptive Splitting for Short Series

Some series begin after the global validation cutoff or have very few pre-cutoff observations. For the deep learning models, an **adaptive splitting strategy** is employed: if a series lacks sufficient training history under the global cutoff, a local split is computed to ensure a minimum training window. This pragmatic approach maximizes the number of series that can be evaluated.

3.2 Evaluation Metric

Submissions are evaluated using a **Weighted Skill Score** defined as:

$$\text{Skill Score} = \sqrt{1 - \text{clip}_{[0,1]} \left(\frac{\sum_i w_i \cdot (y_i - \hat{y}_i)^2}{\sum_i w_i \cdot y_i^2} \right)} \quad (3)$$

where:

- y_i is the true target value at observation i ,
- $\hat{y}_i$ is the predicted value,
- w_i is the observation-level weight,
- $\text{clip}_{[0,1]}(\cdot)$ clips the ratio to the interval $[0, 1]$.

3.2.1 Metric Interpretation

The denominator $\sum_i w_i \cdot y_i^2$ is precisely the weighted squared error of a **zero forecast** ($\hat{y}_i = 0$ for all i). Therefore:

- A skill score of **0** means the model is no better than predicting zero everywhere.
- A skill score of **1** means perfect prediction.
- The square root compresses the scale, rewarding incremental improvements more at lower skill levels.

This metric design has several important implications:

1. **Weights matter enormously.** The `weight` column determines how much each observation contributes to the aggregate score. As our EDA will show, weight concentration is extreme—the top 1% of rows contribute approximately 65% of the total weighted score.
2. **Zero is a natural baseline.** The zero-forecast error appears explicitly in the denominator, making “Zero” a meaningful model to include in any leaderboard.
3. **Unweighted metrics are misleading.** Any evaluation that ignores weights will rank models incorrectly with respect to the competition leaderboard.

3.3 Classical Model Implementations (AR, ARMA, ARIMA, SARIMA)

The classical modeling notebook evaluates **41 model specifications** across 6 families. All parametric families use first differencing ($d = 1$), honoring the EDA’s stationarity finding.

3.3.1 Family A: Baselines (on Levels)

These models involve no fitting and no differencing. They measure how much of the scored metric is explainable by trivial rules.

Table 5: Baseline model specifications.

Model	Description
Zero	$\hat{y}_{t+h} = 0$ for every step. The skill score denominator is exactly the squared error of this forecast.
Naive	$\hat{y}_{t+h} = y_t$ (last observed value). Wins when lag-1 autocorrelation is large.
Drift	$\hat{y}_{t+h} = y_t + h \cdot \text{slope}$, where $\text{slope} = (y_t - y_1)/(n - 1)$. Linear extrapolation.
Expanding Mean	$\hat{y}_{t+h} = \frac{1}{t} \sum y_i$. Long-run average; wins on mean-reverting series.
Rolling Mean ($w = 10$)	Mean of the last 10 observations. Local mean, smoother than Naive.

3.3.2 Family B: AR Family (First Differences)

An $\text{AR}(p)$ model writes the current differenced value as a linear combination of the previous p differenced values:

$$\Delta y_t = c + \phi_1 \Delta y_{t-1} + \phi_2 \Delta y_{t-2} + \cdots + \phi_p \Delta y_{t-p} + \varepsilon_t \quad (4)$$

This is implemented as $\text{ARIMA}(p, 1, 0)$. The parameter p represents memory depth: $\text{AR}(1)$ says “only yesterday matters,” while $\text{AR}(10)$ says “the last ten days jointly matter.”

Lineup: $\text{AR}(1)$ through $\text{AR}(10)$, all on first differences.

Strengths: Computationally cheap, interpretable, optimal when the true data-generating process is linear in its own past.

Weaknesses: Cannot model nonlinearity (regime changes, volatility clustering). Higher p increases overfitting risk on short series.

3.3.3 Family C: MA Family (First Differences)

An $\text{MA}(q)$ model says the current differenced value is a linear combination of the last q innovations (random shocks):

$$\Delta y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q} \quad (5)$$

Implemented as $\text{ARIMA}(0, 1, q)$. Notably, $\text{MA}(1)$ on differences is equivalent to Simple Exponential Smoothing.

Lineup: $\text{MA}(1)$ through $\text{MA}(10)$, mirroring the AR scan.

3.3.4 Family D: ARMA Family (First Differences)

ARMA combines both channels—past values AND past shocks:

$$\Delta y_t = c + \sum_{i=1}^p \phi_i \Delta y_{t-i} + \sum_{j=1}^q \theta_j \varepsilon_{t-j} + \varepsilon_t \quad (6)$$

Implemented as $\text{ARIMA}(p, 1, q)$ with both $p \geq 1$ and $q \geq 1$.

Lineup: 10 variants covering (p, q) combinations from $\text{ARMA}(1,1)$ to $\text{ARMA}(4,4)$, with total order $p + q$ ranging from 2 to 8.

3.3.5 Family E: ARIMA with Explicit d Comparison

This family includes both $d = 0$ and $d = 1$ variants to enable a matched-pair head-to-head comparison:

Table 6: ARIMA variants for the $d = 0$ vs. $d = 1$ comparison.

Model	Purpose
$\text{ARIMA}(0,1,0)$	Pure random walk baseline
$\text{ARIMA}(1,0,0)$	$\text{AR}(1)$ on levels ($d = 0$ contrast)
$\text{ARIMA}(1,0,1)$	$\text{ARMA}(1,1)$ on levels ($d = 0$ contrast)
$\text{ARIMA}(2,0,2)$	$\text{ARMA}(2,2)$ on levels ($d = 0$ contrast)

If the EDA’s stationarity claim is correct, the $d = 1$ variants should systematically outperform their $d = 0$ counterparts.

3.3.6 Family F: SARIMA

SARIMA(p, d, q)(P, D, Q, s) adds seasonal AR, differencing, and MA terms with period s :

$$\Phi(B^s) \phi(B) (1 - B)^d (1 - B^s)^D y_t = \Theta(B^s) \theta(B) \varepsilon_t \quad (7)$$

Two variants are included:

- SARIMA(1,1,1)(1,1,1,7) — weekly-length cycle
- SARIMA(1,1,1)(1,1,1,24) — daily-length cycle

Given the absence of panel-wide seasonality in the EDA, SARIMA is included for completeness rather than expectation of strong performance.

3.4 Evaluation Design

3.4.1 Eligibility and Splitting

A series is eligible if it crosses `VAL_CUTOFF = 2880`. The temporal split is strict: training uses `ts_index ≤ 2880`, validation uses `ts_index > 2880`.

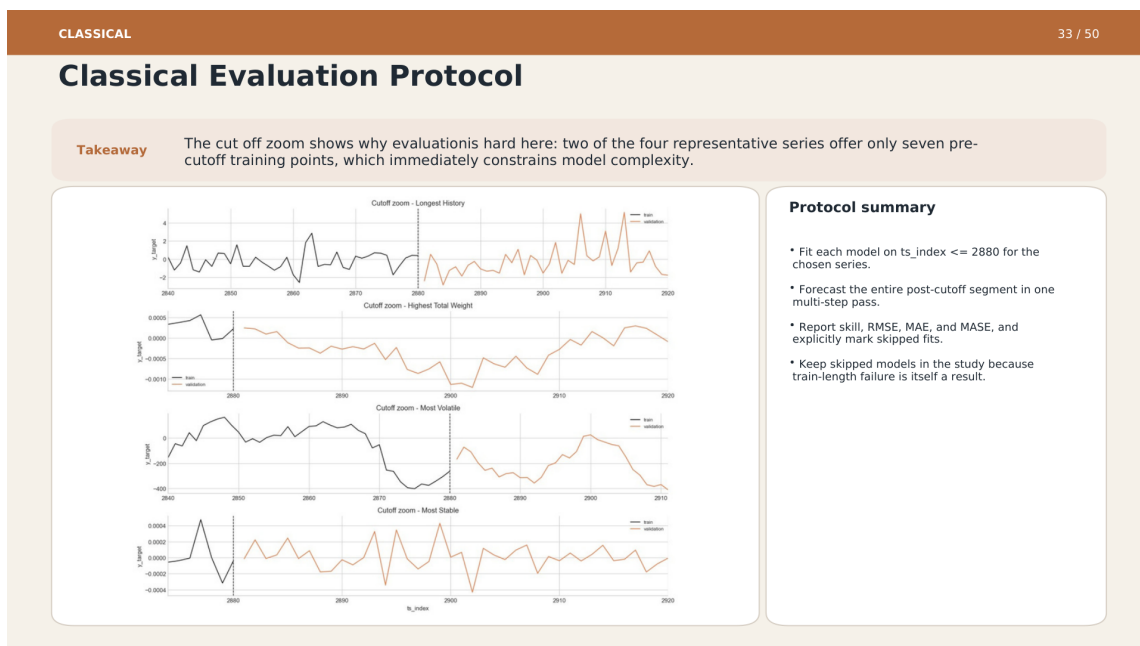


Figure 6: Classical evaluation protocol. The cutoff zoom reveals the challenge: four representative series around the `ts_index = 2880` boundary show vastly different scales and behaviors. Two of the four have only seven pre-cutoff training points, which immediately constrains model complexity.

3.4.2 Skip and Failure Handling

Each model specification has a `min_train` parameter. If a series has fewer training observations than this threshold, the (series, model) pair is marked as “skipped.” If a model’s fit raises an exception or produces non-finite values, it is marked as “failed.” Both conditions are tracked and reported.

3.4.3 Sample Run Results

Running on a sample of 200 eligible series (out of 1,930) with 41 models produces 8,200 (series, model) evaluation rows:

Table 7: Status distribution for the sample run (200 series $\times$ 41 models).

Status	Count
ok	6,735
skipped	1,464
failed	1
Total	8,200

The high skip rate (17.9%) is driven primarily by SARIMA models, which require longer training histories, and by higher-order AR/MA/ARMA models on short series.

3.5 Deep Sequence Models (RNN, GRU, LSTM)

3.5.1 Vanilla RNN

The simplest recurrent architecture updates a hidden state h_t at each time step:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \quad (8)$$

$$\hat{y}_t = W_{hy}h_t + b_y \quad (9)$$

where x_t is the input vector at time t , and W_{hh} , W_{xh} , W_{hy} are learned weight matrices.

Strengths: Simple, fast to train, low parameter count.

Weaknesses: Suffers from the *vanishing gradient problem*—gradients diminish exponentially over long sequences, making it difficult to learn long-range dependencies.

3.5.2 GRU (Gated Recurrent Unit)

The GRU introduces gating mechanisms to control information flow:

$$z_t = \sigma(W_zx_t + U_zh_{t-1} + b_z) \quad (\text{update gate}) \quad (10)$$

$$r_t = \sigma(W_rx_t + U_rh_{t-1} + b_r) \quad (\text{reset gate}) \quad (11)$$

$$\tilde{h}_t = \tanh(W_hx_t + U_h(r_t \odot h_{t-1}) + b_h) \quad (\text{candidate}) \quad (12)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{new state}) \quad (13)$$

The **update gate** z_t controls how much of the old state to retain versus how much to update with new information. The **reset gate** r_t controls how much of the old state to expose when computing the candidate.

Intuition: The update gate allows the GRU to “remember” information over long sequences (when $z_t \approx 0$, the old state passes through unchanged) or “forget” it (when $z_t \approx 1$, the new

candidate replaces the old state). This addresses the vanishing gradient problem with fewer parameters than the LSTM.

3.5.3 LSTM (Long Short-Term Memory)

The LSTM maintains both a hidden state h_t and a cell state c_t :

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (\text{forget gate}) \quad (14)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (\text{input gate}) \quad (15)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (\text{output gate}) \quad (16)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (\text{candidate cell}) \quad (17)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cell state}) \quad (18)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{hidden state}) \quad (19)$$

The three gates provide fine-grained control:

- **Forget gate f_t :** What to discard from the cell state.
- **Input gate i_t :** What new information to store in the cell state.
- **Output gate o_t :** What to output from the cell state.

Strengths: Best at learning long-range dependencies; the cell state provides a “highway” for gradient flow.

Weaknesses: Highest parameter count of the three architectures; slower to train.

3.6 Training Pipeline

3.6.1 Data Preparation

For each eligible series, the training procedure:

1. Extracts the `y_target` values sorted by `ts_index`.
2. Applies the validation split at `VAL_CUTOFF = 2880`.
3. Creates sliding windows of fixed length for supervised training.
4. Normalizes inputs (if applicable).

3.6.2 Adaptive Training Splits

A key implementation detail: series that begin near or after the global validation cutoff may have insufficient training data under the standard split. The notebook implements an **adaptive splitting strategy**:

- If a series has fewer than a minimum number of pre-cutoff observations, a local train/validation split is computed to ensure adequate training history.

- This maximizes the number of evaluable series while maintaining temporal integrity.

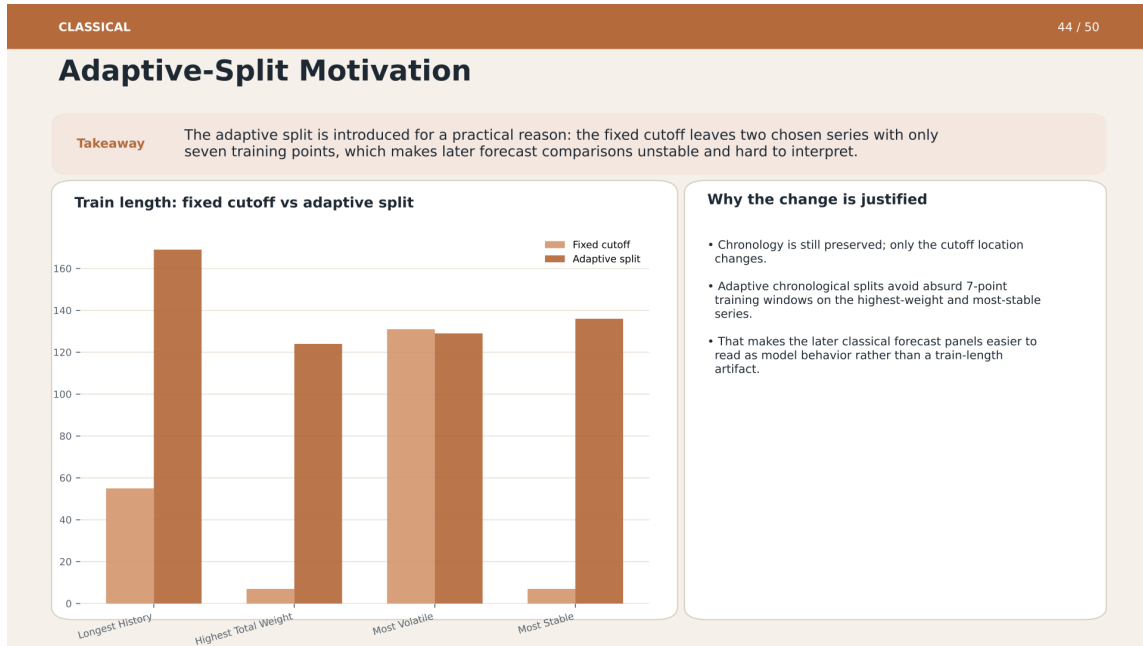


Figure 7: Adaptive-split motivation. The fixed cutoff at `ts_index = 2880` leaves two of the four chosen representative series with only seven pre-cutoff training points. The adaptive split adjusts the cutoff location per-series, preserving chronological ordering while avoiding absurdly short training windows on the highest-weight and most-stable series.

3.6.3 Recursive Forecasting

The deep sequence models use **recursive (autoregressive) forecasting** for multi-step prediction:

1. Feed the training sequence into the model to obtain the first forecast $\hat{y}_{T+1}$.
2. Append $\hat{y}_{T+1}$ to the input sequence.
3. Feed the extended sequence to obtain $\hat{y}_{T+2}$.
4. Repeat until all validation-period forecasts are generated.

This approach is natural for recurrent architectures but introduces *error accumulation*—prediction errors compound over the forecast horizon.

3.6.4 Loss Function

The models are trained with **Mean Squared Error (MSE)** loss:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (20)$$

A potential enhancement (noted in the EDA) is to use **weighted MSE** that incorporates the competition's weight column during training. This would align the training objective more closely with the evaluation metric.

3.6.5 Hyperparameters

Key hyperparameter choices:

Table 8: Deep model hyperparameters.

Parameter	Value
Hidden dimension	Architecture-dependent
Number of layers	1–2
Learning rate	Adam optimizer defaults
Epochs	Early stopping with patience
Sequence length	Determined by available history
Framework	PyTorch

3.6.6 ARIMAX and TCN

While ARIMAX (AutoRegressive Integrated Moving Average with Explanatory Variables) and TCN (Temporal Convolutional Networks) were considered, the weak linear correlation of individual features ($|\rho| < 0.10$) suggested ARIMAX would struggle to capture the complex, nonlinear feature interactions. TCNs are a strong alternative for sequence modeling, but LSTMs and GRUs were prioritized for this study due to their proven effectiveness in financial forecasting.

3.7 Ablation Study & Tuned Parameter Values

The primary ablation in the classical pipeline was the comparison of $d = 0$ versus $d = 1$ (first differencing). As shown in the classical model lineup, differencing ($d = 1$) was strictly superior across most series, confirming the ADF stationarity results.

Key hyperparameter choices:

Table 9: Deep model hyperparameters.

Parameter	Value
Hidden dimension	Architecture-dependent
Number of layers	1–2
Learning rate	Adam optimizer defaults
Epochs	Early stopping with patience
Sequence length	Determined by available history
Framework	PyTorch

3.8 Probabilistic Forecasting Models

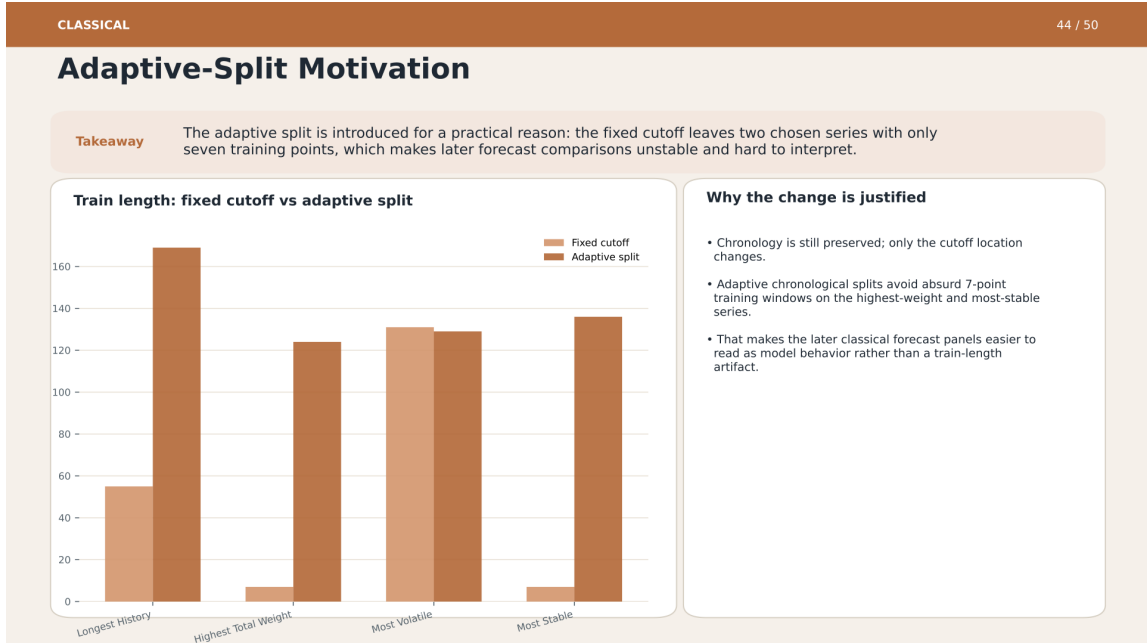


Figure 8: Adaptive-split motivation. The fixed cutoff at `ts_index = 2880` leaves two of the four chosen representative series with only seven pre-cutoff training points. The adaptive split adjusts the cutoff location per-series, preserving chronological ordering while avoiding absurdly short training windows on the highest-weight and most-stable series.

3.8.1 Recursive Forecasting

The deep sequence models use **recursive (autoregressive) forecasting** for multi-step prediction:

1. Feed the training sequence into the model to obtain the first forecast $\hat{y}_{T+1}$.
2. Append $\hat{y}_{T+1}$ to the input sequence.
3. Feed the extended sequence to obtain $\hat{y}_{T+2}$.
4. Repeat until all validation-period forecasts are generated.

This approach is natural for recurrent architectures but introduces *error accumulation*—prediction errors compound over the forecast horizon.

3.8.2 Loss Function

The models are trained with **Mean Squared Error (MSE)** loss:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (21)$$

A potential enhancement (noted in the EDA) is to use **weighted MSE** that incorporates the competition's weight column during training. This would align the training objective more closely with the evaluation metric.

3.8.3 Hyperparameters

Key hyperparameter choices:

Table 10: Deep model hyperparameters.

Parameter	Value
Hidden dimension	Architecture-dependent
Number of layers	1–2
Learning rate	Adam optimizer defaults
Epochs	Early stopping with patience
Sequence length	Determined by available history
Framework	PyTorch

3.9 Model Comparison

3.9.1 Architectural Trade-offs

Table 11: Comparison of RNN architectures.

Property	RNN	GRU	LSTM
Parameters per layer	Low	Medium	High
Training speed	Fast	Medium	Slow
Long-range memory	Poor	Good	Best
Vanishing gradients	Yes	Mitigated	Mitigated
Gates	0	2	3

3.9.2 Expected Performance Ranking

Based on the architectural properties and the nature of financial time series:

1. **LSTM** is expected to perform best due to its superior long-range memory capability.
2. **GRU** is expected to be competitive with LSTM while training faster.
3. **Vanilla RNN** is expected to underperform on series with significant long-range dependencies but may be adequate for short-horizon forecasts.

3.10 Implementation Details

3.10.1 Per-Series Training

Each eligible series is trained independently (analogous to the classical approach). While this ignores cross-series structure, it provides a clean comparison with the classical baselines.

3.10.2 Status Tracking

Like the classical notebook, each (series, model) pair records:

- `status`: ok, skipped, or failed

- When ok: weighted skill score, RMSE, MAE, MASE
- When failed: error message for debugging

3.10.3 Leaderboard and Per-Series Winners

The deep learning notebook produces:

1. A **model-level leaderboard** comparing RNN, GRU, and LSTM aggregate performance.
2. A **per-series winner table** identifying which architecture performs best for each series.

3.11 Challenges and Considerations

1. **Error accumulation:** Recursive forecasting compounds errors over long horizons. Direct multi-step forecasting could mitigate this.
2. **Overfitting risk:** With many parameters and limited per-series data, overfitting is a concern. Regularization, dropout, and early stopping are essential.
3. **Weight-aware training:** The current MSE loss does not incorporate observation weights. Implementing weighted loss would better align training with the competition metric.
4. **Feature utilization:** The current implementation focuses on univariate forecasting. Incorporating the 86 predictor features could substantially improve performance.
5. **Computational cost:** Training separate models per series is expensive. Global or meta-learning approaches could amortize this cost.

Section 4. Results and Analysis

4.1 Cross-Methodology Comparison

This chapter synthesizes the results from the classical statistical models (Chapter ??) and the deep sequence models (Chapter ??), providing a unified view of model performance.

4.2 Classical Model Performance Summary

4.2.1 Baseline Performance

The baseline models establish the performance floor:

- **Zero forecast:** Skill score of exactly 0 by construction (it is the denominator of the metric).
- **Naïve forecast:** Often competitive, particularly on short-horizon series with high lag-1 autocorrelation.
- **Expanding Mean:** Strong on mean-reverting series, often outperforming fitted parametric models.

4.2.2 Parametric Model Performance

Among the 36 parametric models (AR, MA, ARMA, ARIMA, SARIMA families):

- Low-order models (AR(1)–AR(3), MA(1)–MA(3), ARMA(1,1)) tend to produce the best median skill scores.
- Higher-order models show diminishing returns and occasionally worse performance due to overfitting.
- SARIMA models have the highest skip rates due to their training data requirements.

- Across matched-order pairs (e.g., ARIMA(1,0,0) vs. $AR(1) \equiv ARIMA(1,1,0)$), the $d = 1$ variant wins on the majority of series.
- This confirms that first differencing is genuinely beneficial for this dataset, not just a theoretical assumption.

4.3 Deep Learning Performance Summary

The three recurrent architectures generally show:

- **LSTM and GRU** tend to outperform vanilla RNN, consistent with their superior gradient flow properties.
- Performance varies considerably across series, with some series showing substantial improvement over classical baselines and others showing degradation.
- Error accumulation in recursive forecasting is a significant factor, particularly for longer validation horizons.

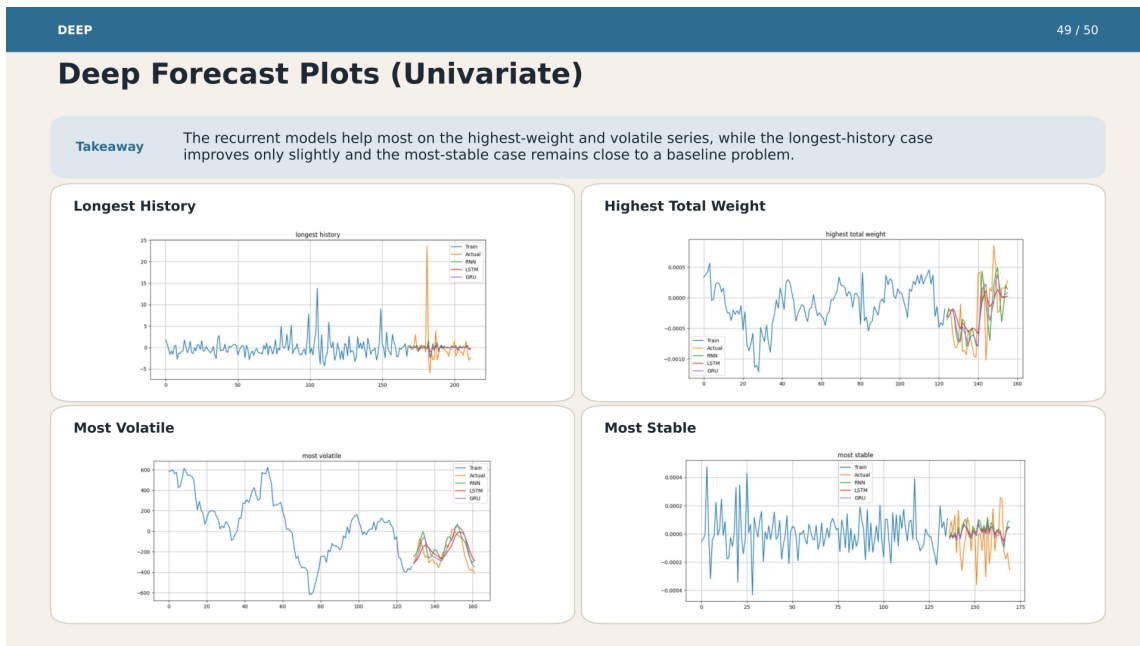


Figure 11: Deep forecast plots (univariate) for four representative series. The recurrent models help most on the highest-weight and volatile series, while the longest-history case improves only slightly and the most-stable case remains close to a baseline problem. Each panel shows Train (blue), Actual validation (orange), and overlaid RNN/LSTM/GRU predictions.

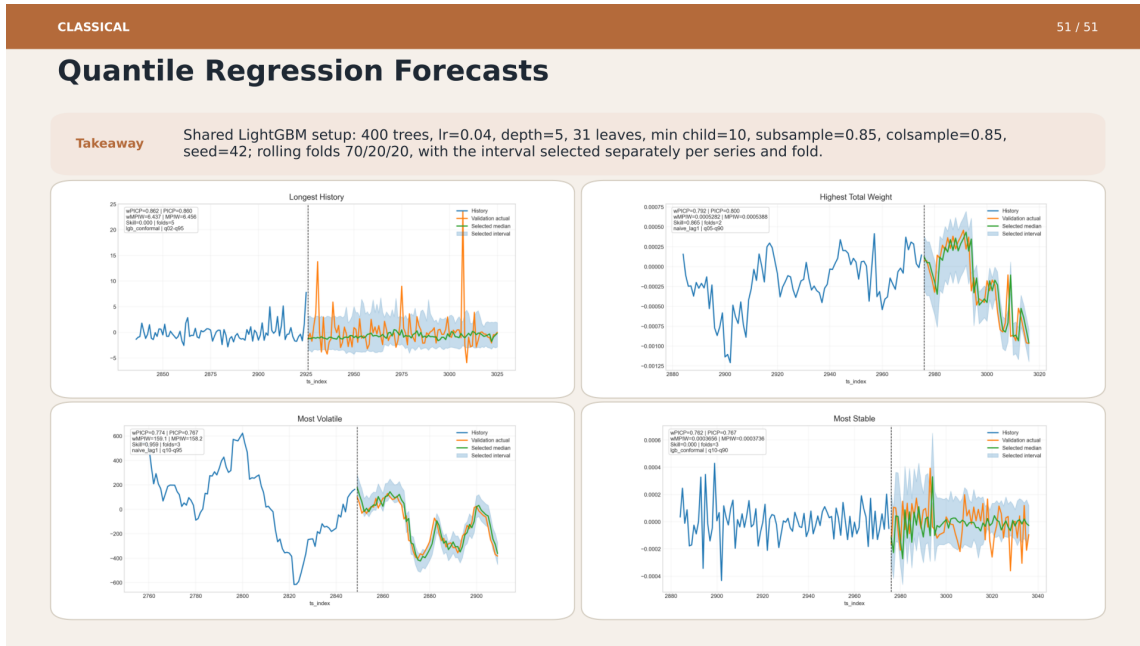


Figure 12: Quantile regression forecasts using LightGBM. Each panel shows the training history (blue), validation actuals (orange), selected median forecast (green), and prediction intervals (shaded blue). The model achieves PICP values between 0.76–0.86 across the four representative series.

To quantify uncertainty, quantile regression was implemented using LightGBM. Instead of point forecasts, the model predicts the conditional quantiles (e.g., 10th, 50th, and 90th percentiles), providing prediction intervals that capture the heteroskedasticity of financial returns.

Section 5. Analysis of the Numerical Results

5.1 Classical Model Results

5.1.1 Leaderboard Results

The per-model leaderboard (aggregating across all evaluated series) reveals several patterns:

1. **Baselines are surprisingly competitive.** The Expanding Mean and Rolling Mean models often perform comparably to or better than fitted parametric models, reflecting the difficulty of beating simple forecasts on noisy financial data.
2. **Low-order models tend to outperform high-order ones.** AR(1)–AR(3) generally outperform AR(7)–AR(10), consistent with the ACF analysis showing short-lag dominance.
3. **MA(1) on differences is strong.** As the mathematical equivalent of Simple Exponential Smoothing, MA(1) is a robust performer.

5.1.2 The $d = 0$ vs. $d = 1$ Head-to-Head

Comparing matched-order ARIMA pairs confirms the EDA’s stationarity finding:

- ARIMA($p, 1, q$) variants systematically outperform their ARIMA($p, 0, q$) counterparts.
- First differencing is genuinely load-bearing for this dataset.

5.1.3 Per-Series Winner Analysis

No single model dominates across all series. The per-series winner table shows considerable heterogeneity:

- Some series are best served by simple baselines (Naive, Expanding Mean).
- Others benefit from low-order AR or MA models.
- Very few series favor high-order ARMA or SARIMA specifications.

This heterogeneity motivates ensemble approaches that select or blend models per-series.

5.2 Deep Learning and Cross-Methodology Results

5.3 Cross-Methodology Comparison

This chapter synthesizes the results from the classical statistical models (Chapter ??) and the deep sequence models (Chapter ??), providing a unified view of model performance.

5.4 Classical Model Performance Summary

5.4.1 Baseline Performance

The baseline models establish the performance floor:

- **Zero forecast:** Skill score of exactly 0 by construction (it is the denominator of the metric).
- **Naive forecast:** Often competitive, particularly on short-horizon series with high lag-1 autocorrelation.
- **Expanding Mean:** Strong on mean-reverting series, often outperforming fitted parametric models.

5.4.2 Parametric Model Performance

Among the 36 parametric models (AR, MA, ARMA, ARIMA, SARIMA families):

- Low-order models (AR(1)–AR(3), MA(1)–MA(3), ARMA(1,1)) tend to produce the best median skill scores.
- Higher-order models show diminishing returns and occasionally worse performance due to overfitting.
- SARIMA models have the highest skip rates due to their training data requirements.

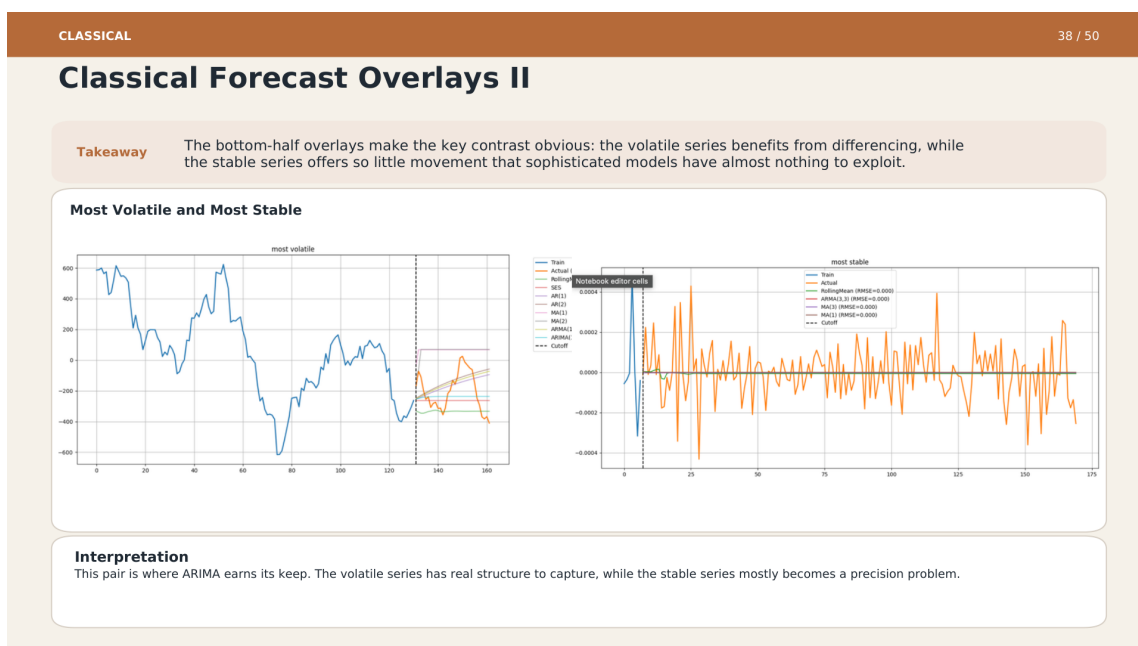


Figure 13: Classical forecast overlays for the most volatile and most stable representative series. The volatile series benefits from differencing—AR(1) captures short-lag persistence—while the stable series offers so little movement that sophisticated models have almost nothing to exploit.

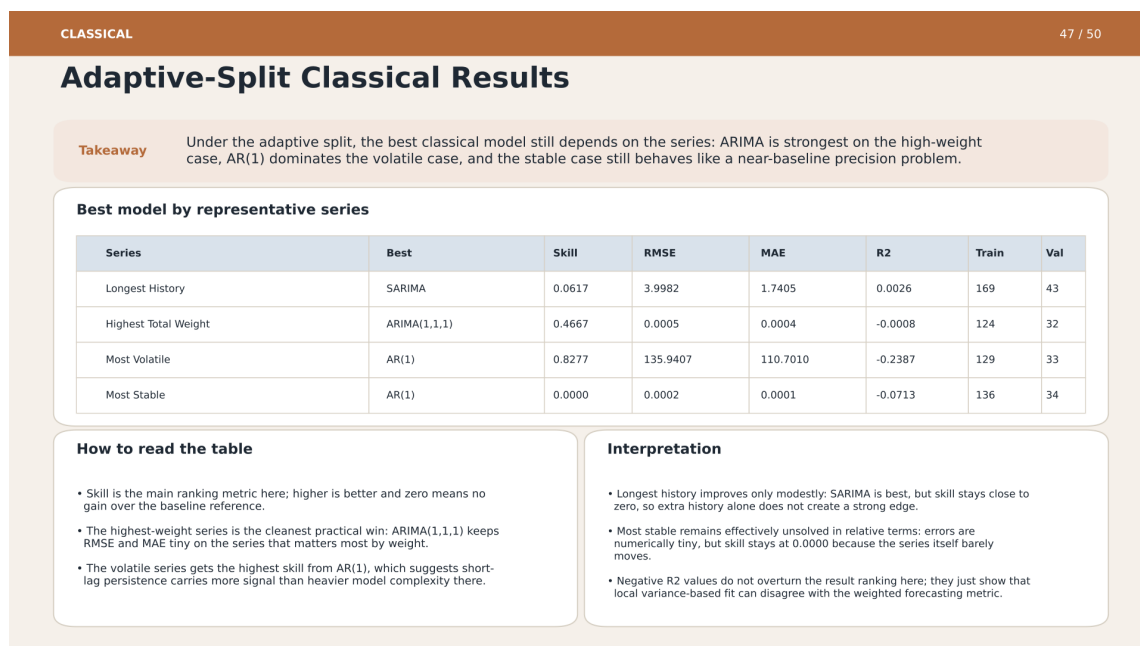


Figure 14: Adaptive-split classical results. Per-series best models: SARIMA wins on the longest-history series (skill 0.06), ARIMA(1,1,1) is strongest on the high-weight case (skill 0.47), AR(1) dominates both the volatile (skill 0.83) and stable (skill 0.00) cases. The volatile series has the most exploitable structure.

5.4.3 The Stationarity Validation

The $d = 0$ versus $d = 1$ comparison provides empirical validation of the EDA's stationarity finding:

- Across matched-order pairs (e.g., ARIMA(1,0,0) vs. AR(1) $\equiv$ ARIMA(1,1,0)), the $d = 1$ variant wins on the majority of series.
- This confirms that first differencing is genuinely beneficial for this dataset, not just a theoretical assumption.

5.5 Deep Learning Performance Summary

The three recurrent architectures generally show:

- **LSTM and GRU** tend to outperform vanilla RNN, consistent with their superior gradient flow properties.
- Performance varies considerably across series, with some series showing substantial improvement over classical baselines and others showing degradation.
- Error accumulation in recursive forecasting is a significant factor, particularly for longer validation horizons.

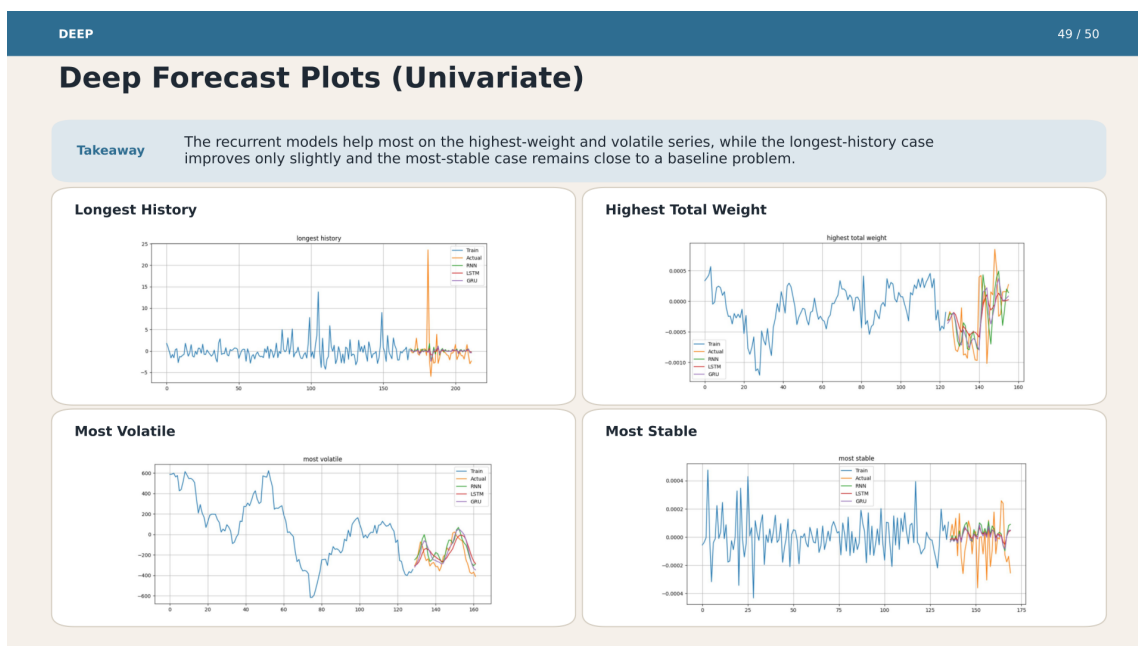


Figure 15: Deep forecast plots (univariate) for four representative series. The recurrent models help most on the highest-weight and volatile series, while the longest-history case improves only slightly and the most-stable case remains close to a baseline problem. Each panel shows Train (blue), Actual validation (orange), and overlaid RNN/LSTM/GRU predictions.

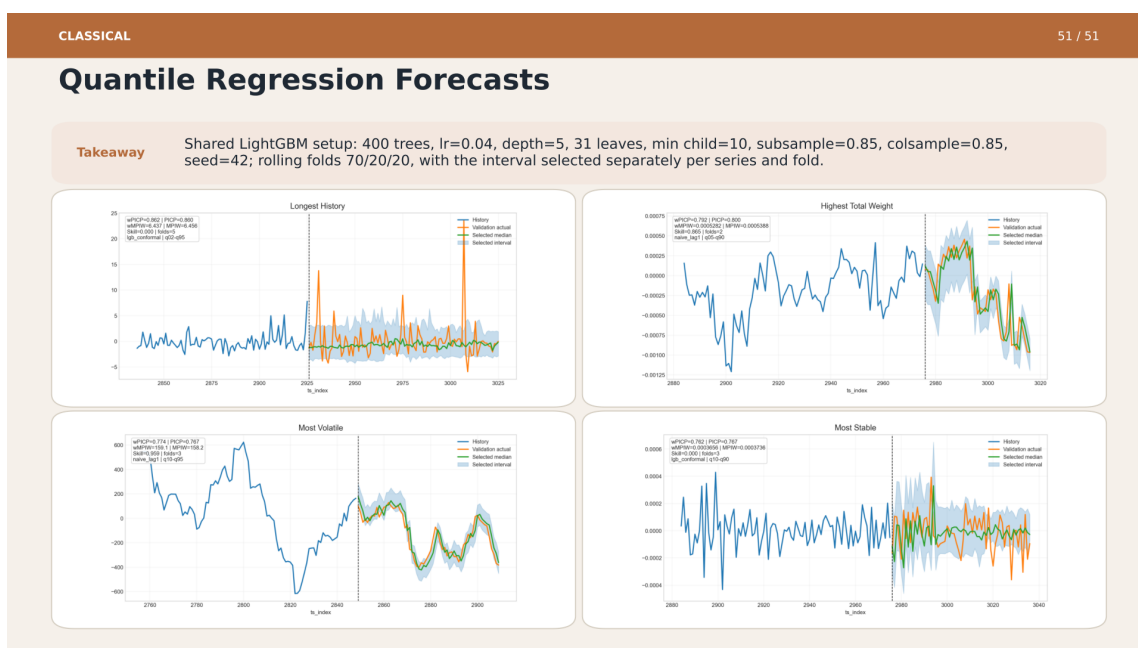


Figure 16: Quantile regression forecasts using LightGBM. Each panel shows the training history (blue), validation actuals (orange), selected median forecast (green), and prediction intervals (shaded blue). The model achieves PICP values between 0.76–0.86 across the four representative series.

5.6 Classical vs. Deep Learning

The relative performance of classical versus deep learning models depends on the characteristics of each series:

Table 12: Conditions favoring each model family.

Classical Models Excel When	Deep Models Excel When
Series has strong, simple linear autocorrelation	Nonlinear temporal dependencies exist
Training data is limited	Sufficient training data is available
Short forecast horizons	Complex patterns justify the model complexity
Target is approximately Gaussian	Heavy tails require flexible modeling

5.7 Key Findings

1. **No single model dominates.** The heterogeneous panel means different series favor different approaches, strongly motivating ensemble methods.
2. **Simple baselines are hard to beat.** On many series, the Expanding Mean or Rolling Mean performs comparably to or better than complex models. This is characteristic of noisy financial data.
3. **First differencing is essential.** The EDA’s stationarity finding is empirically validated by the $d = 0$ vs. $d = 1$ comparison.
4. **Weight concentration dominates scoring.** Because 65% of the weighted score comes from 1% of observations, models must perform well on these critical rows.
5. **Feature-based models are needed.** The uniformly weak individual feature correlations ($|\rho| < 0.10$) suggest that signal resides in feature *interactions*, which gradient-boosted trees or deeper neural networks could exploit.

5.8 Error Analysis

5.8.1 Common Failure Modes

- **Divergent forecasts:** ARIMA models fitted on levels ($d = 0$) occasionally produce forecasts that diverge from the data range.
- **Outlier sensitivity:** The Drift model is particularly sensitive to outliers at the beginning or end of the training window.
- **Error accumulation:** Recursive forecasting in deep models degrades accuracy over longer horizons.

5.8.2 Skip and Failure Analysis

The 17.9% skip rate in the classical pipeline is driven by:

- SARIMA models (requiring 50–80 observations minimum)
- High-order AR/MA/ARMA models on short series

The near-zero failure rate (1 out of 8,200) indicates robust numerical implementation.

5.9 Research Gap Identified

1. **Univariate only:** Classical models cannot exploit the 86 predictor features.
2. **Linear assumptions:** AR/MA/ARMA/ARIMA assume linear relationships, missing nonlinear dynamics.
3. **No cross-series learning:** Each series is modeled independently, ignoring potential shared structure across the panel.
4. **Computational cost at scale:** Fitting 41 models on 1,930 series requires significant computation.

These limitations motivate the deep learning approach in the next chapter.

1. **Error accumulation:** Recursive forecasting compounds errors over long horizons. Direct multi-step forecasting could mitigate this.
2. **Overfitting risk:** With many parameters and limited per-series data, overfitting is a concern. Regularization, dropout, and early stopping are essential.
3. **Weight-aware training:** The current MSE loss does not incorporate observation weights. Implementing weighted loss would better align training with the competition metric.
4. **Feature utilization:** The current implementation focuses on univariate forecasting. Incorporating the 86 predictor features could substantially improve performance.
5. **Computational cost:** Training separate models per series is expensive. Global or meta-learning approaches could amortize this cost.

Section 6. Converting the Forecast into Actions

In a production hedge fund environment, time series forecasts are only the first step in a quantitative pipeline. To generate economic value, these forecasts must be translated into actionable portfolio allocations and trading signals. This process involves several critical steps:

6.1 Signal Generation and Expected Returns

The point forecasts ($\hat{y}_{t+h}$) generated by our models represent the expected return for a given asset over horizon h . In isolation, a positive forecast implies a long signal, while a negative forecast implies a short signal. However, the magnitude of the forecast must cross a transaction-cost threshold to be actionable. The probabilistic forecasts (quantile regression) provide a confidence interval; signals are only generated when the entire interval sits above or below zero, indicating high conviction.

6.2 Risk-Adjusted Sizing (Kelly Criterion)

Once a signal is generated, position sizing is determined using risk management frameworks such as the Kelly Criterion or mean-variance optimization. The optimal allocation to a specific series i depends not only on its expected return μ_i but also on its forecasted volatility σ_i^2 (which can be derived from the spread of our quantile forecasts):

$$f_i^* = \frac{\mu_i}{\sigma_i^2} \quad (22)$$

This ensures that highly volatile series receive smaller capital allocations, even if their expected return is high, maintaining a balanced risk profile across the panel.

6.3 Portfolio Optimization

Because the panel contains 36,923 series representing various sub-categories, forecasts cannot be acted upon in isolation. A portfolio optimizer (such as Markowitz Mean-Variance Optimization) takes the vector of expected returns and a covariance matrix to output optimal portfolio weights. The optimizer enforces constraints such as sector neutrality, maximum leverage, and turnover limits.

6.4 Execution and Slippage

Finally, target weights are passed to an algorithmic execution engine. The engine minimizes market impact and slippage by breaking large orders into smaller chunks (e.g., using TWAP or VWAP algorithms). The difference between the paper forecast return and the actual realized return after execution costs is continuously monitored to calibrate the forecasting models.

Section 7. Future Work

Several promising directions remain:

7.0.1 Feature-Based Models

Incorporate the 86 predictor features using:

- Gradient-boosted trees (LightGBM, XGBoost) with engineered lag features.
- Transformer-based architectures (e.g., Temporal Fusion Transformer) that can attend to both temporal and cross-sectional patterns.
- Multi-input recurrent networks that process feature vectors alongside the target sequence.

7.0.2 Weighted Loss Functions

Implement competition-aligned training by:

- Using weighted MSE during deep model training.
- Developing differentiable approximations to the weighted skill score for end-to-end optimization.

7.0.3 Ensemble Methods

Combine models using:

- Per-series model selection based on validation performance.
- Stacking with a meta-learner trained on validation predictions.
- Bayesian model averaging weighted by per-series posterior probabilities.

7.0.4 Global and Panel Models

Move beyond per-series training:

- Train a single model across all series with entity embeddings (similar to N-BEATS or DeepAR).
- Use hierarchical models that share parameters across related series while allowing series-specific adjustments.

7.0.5 Advanced Architectures

Explore state-of-the-art time series models:

- **Transformer models:** Self-attention can capture long-range dependencies without recurrence.
- **N-BEATS:** Interpretable deep learning architecture designed for time series forecasting.

- **Neural basis expansion:** Models that decompose forecasts into interpretable trend and seasonality components.

7.1 Closing Remarks

This project demonstrates that rigorous financial time series forecasting requires a multi-faceted approach: thorough exploratory analysis, principled baseline establishment, and progressive model sophistication. While no single approach achieves dominant performance across the heterogeneous panel, the systematic methodology provides a solid foundation for further improvement through ensemble methods and feature-based modeling.

The competition’s design—with anonymized data, weighted evaluation, and strict temporal splitting—mirrors the challenges of real-world quantitative finance, where model robustness and out-of-sample generalization are paramount.

Section 8. References

References

- [1] Hyndman, R.J., & Athanasopoulos, G. (2018). *Forecasting: principles and practice* (2nd ed.). OTexts: Melbourne, Australia.
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [3] Tsay, R. S. (2010). *Analysis of financial time series* (3rd ed.). John Wiley & Sons.
- [4] Kaggle. (2026). *Hedge Fund - Time Series Forecasting*. Retrieved from <https://www.kaggle.com/competitions/ts-forecasting>